

LISTA DE EXERCÍCIOS 2.6

1. Foi comentado que uma possibilidade para lidar com problemas de seção crítica seria desabilitar as interrupções do sistema. Entretanto, efetuar esse procedimento com frequência poderia prejudicar o relógio do sistema. Explique por que isso acontece e como esses efeitos poderiam ser minimizados.

Resposta: O relógio do sistema é atualizado a cada interrupção de clock. Se as interrupções forem desabilitadas – particularmente por um longo período de tempo – é possível que o relógio do sistema perca o tempo correto. O relógio do sistema também é usado para escalonamento. Por exemplo, o quantum para um processo é expresso como um número de pulsos de relógio (*clock ticks*). A cada interrupção de relógio, o escalonador determina se o quantum para o processo em execução no momento expirou. Se as interrupções de *clock* fossem desabilitadas, o escalonador poderia atribuir de maneira não precisa os quanta. Este efeito pode ser minimizado desabilitando interrupções de *clock* somente por períodos de tempo muito curtos.

2. Explique o conceito de atomicidade de transação.

Resposta: Uma transação pode ser vista como uma série de operações de leitura (*read*) e escrita (*write*) até que algum dado seja seguido por uma operação de *commit*. Se a série de operações em uma transação não pode ser completada, a transação deve ser abortada e as operações que ocorreram devem ser revertidas (*rolled back*). É importante que essa série de operações em uma transação apareça como uma operação indivisível para assegurar a integridade dos dados sendo atualizados. De outra forma, os dados poderiam ser comprometidos se operações de duas (ou mais) transações diferentes forem sobrepostas (*intermixed*).

3. Qual é o significado do termo “espera ocupada” (*busy wating*)? Que outros tipos de espera existem em um SO? A espera ocupada pode ser evitada? Explique.

Resposta: Espera ocupada (*busy wating*) significa que um processo está esperando que uma condição seja satisfeita em um laço fechado sem liberar o processador. Por outro lado, um processo poderia esperar liberando o processador, e bloqueando em uma condição e esperar ser acordado em algum momento apropriado no futuro. Espera ocupada pode ser evitada, mas incorre na sobrecarga associada com a colocação do processo no estado *sleep* e acordando-o (*wake up*) quando o estado apropriado do programa é alcançado.

4. Explique por que *spinlocks* não são apropriados para sistemas uniprocessados, e ainda assim, são frequentemente usados em sistemas multiprocessados.

Resposta: *Spinlocks* não são apropriados para sistemas monoprocesados, pois a condição que tiraria o processo do laço pode ser obtida somente pela execução de um processo diferente. Caso o processo não libere o processador, outros processos não conseguem a oportunidade de ajustar a condição do programa necessária para que o primeiro processo continue. Em um sistema multiprocessado, outros processos executam em outros processadores e assim, modificam o estado do programa de modo a liberar o primeiro processo do laço (i.e. *spinlock*).

5. Verifique cada um dos códigos fontes fornecidos no âmbito da solução de Peterson, mutexes e *test and set* para proteção de seção crítica. Trace comentários entre a proximidade da teoria fornecida em classe e a prática manifestada em cada um dos códigos-fonte exemplo. Caso tenha dúvidas específicas, registre em sua resposta para que sejam tratadas na aula síncrona.